

FPGA experiment report

Task: FPGA 4

Student 1 name: Mahmood Stitia ID: 214032682

Student 2: name: Saleh Khalil ID: 213310485

Part 1 - Status update

Fill in the following table according to your progress at the time of the submission. Each row should refer to one of the modules together with its simulation.

The status column should be filled with one of the following:

- Done - If both are written and simulation passed.
- Partially done - If you started writing the module and simulation but it's not working properly.
- Not started - If you didn't get to that part of the task.

When the status is partially done, please use the notes/comments column to explain in a few more words what is the status of the module and what is the status of the simulation.

Fill here:

Module name/Step	status	notes/comments
Control	Done	TB Passed
Debouncer	Done	
Stopwatch	Done	
Elaborated Design	Done	
Synthesis	Done	
Setting up constraints	Done	
Implementing the design	Done	
FPGA programming and debugging	Done	

Issues:

In this section please describe the current issues you are facing.

- Per each module and simulation, if you receive any errors at the time of submission, please describe which errors and what they mean.

Fill here:

- Control:
- Debouncer:
- Stopwatch:
- Elaborated Design:
- Synthesis:
- Setting up constraints:
- Implementing the design:
- FPGA programming and debugging:

Part 2 - Simulations and answers

In this section, please add the images for each simulation as described in the report. Remember to use annotations and explain in your own words what is shown in each simulation.

In some of the tasks in the report, there are also questions that you should answer here.

Make sure to include images of all simulations running at the time of submission, even if they are not correct.

Fill here:

- **Control:**

2c. Test-bench. The test-bench exhaustively drives the FSM from each of the three reachable states (IDLE, COUNTING, PAUSED) and applies every possible combination of {reset, trig, split} (cii = 0...7).

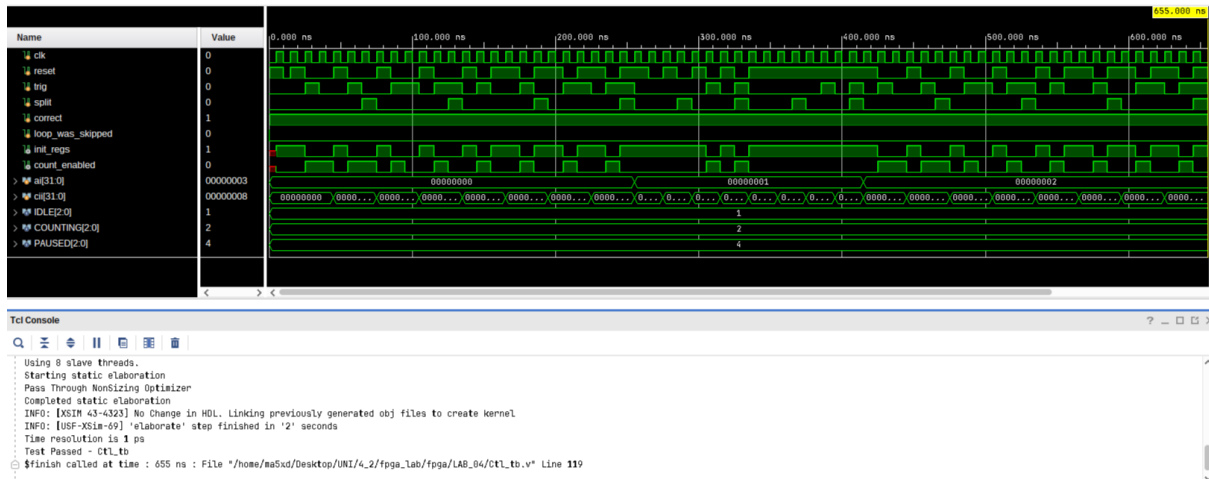


Figure 1: Annotated waveform of Ct1_tb. The outer counter ai sweeps the three starting states (IDLE = 0, COUNTING = 1, PAUSED = 2) and the inner counter cii sweeps the 8 input combinations of {reset, trig, split}. The correct signal stays high for the whole simulation, and the TCL console at the bottom shows Test Passed - Ct1_tb and Corrent was high all the time

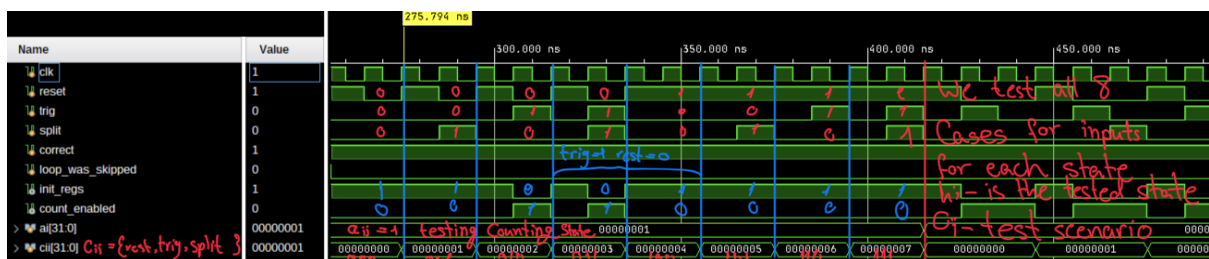


Figure 2: Zoomed view inside the ai = 1 (COUNTING) block. After the short preamble (one reset pulse, then one trig pulse to leave IDLE), the FSM is in COUNTING and count_enabled is high. Each subsequent cii value tests one of the COUNTING transitions in Figure 1: a trig pulse drops count_enabled (move to PAUSED) and a later reset forces init_regs high (return to IDLE).

2d.ii Mealy vs. Moore. The assignment draws the FSM in Mealy form (each arrow shows both inputs and outputs). However, all transitions *leaving the same state* carry the *same output*:

- every arrow leaving IDLE outputs init_regs, count_enabled = 10,
- every arrow leaving COUNTING outputs 01,
- every arrow leaving PAUSED outputs 00.

The output therefore depends only on the current state, so this FSM is in fact a **Moore** FSM. This is exactly how the output is written in Ct1.v:

$$\text{init_regs} = (\text{state} == \text{IDLE}), \quad \text{count_enabled} = (\text{state} == \text{COUNTING}).$$

- **Debouncer:**

3c. High vs. low COUNTER_BITS – the tradeoff.

The hysteresis counter must climb from 0 to $2^{\text{COUNTER_BITS}-1}$ for the MSB to flip and emit a single-cycle pulse. With a 100 MHz clock the debounce window is therefore

$$T_{\text{debounce}} \approx \frac{2^{\text{COUNTER_BITS}-1}}{f_{\text{clk}}}.$$

For the value we use in the top module ($\text{COUNTER_BITS} = 20$), $T_{\text{debounce}} \approx 2^{19}/10^8 \text{ s} \approx 5.24 \text{ ms}$, comfortably above the typical 1–2 ms contact bounce of a tactile button.

The `COUNTER_BITS` parameter determines the duration of the debounce interval. Increasing its value enlarges the counter, which improves immunity to noise and contact bouncing but also introduces longer latency before the system responds to a button press. Conversely, reducing this value shortens response time but increases the likelihood of false triggering caused by unstable or noisy input signals.

This tradeoff is directly analogous to the low-pass-filter-based debouncing approach from earlier experiments. A lower cutoff frequency averages the input over a longer time, suppressing bouncing at the cost of increased delay, while a higher cutoff frequency allows faster response with reduced noise suppression. Similarly, a larger counter value requires longer input stability, whereas a smaller counter value reacts faster but is more sensitive to noise.

- **Stopwatch:**

4a. Block diagram and connectivity.

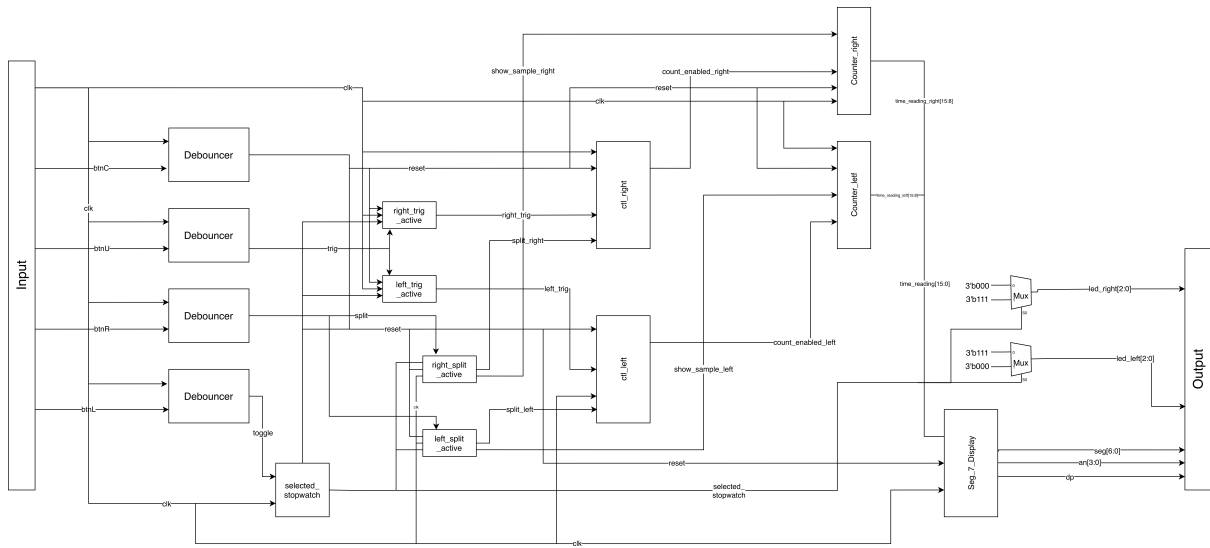


Figure 3: block diagram for the Stopwatch module

Input Signals:

clk	Clock input
btnC	Center Button input
btnU	Upper Button input
btnR	Right Button input
btnL	Left Button input

Output Signals:

seg[6:0]	7-segment display control
an[3:0]	Anode control for multiplexed display
dp	Decimal point control
led_left[2:0]	Left set of LEDs
led_right[2:0]	Right set of LEDs

- **Elaborated Design:**

5b. Elaborated-design schematic.

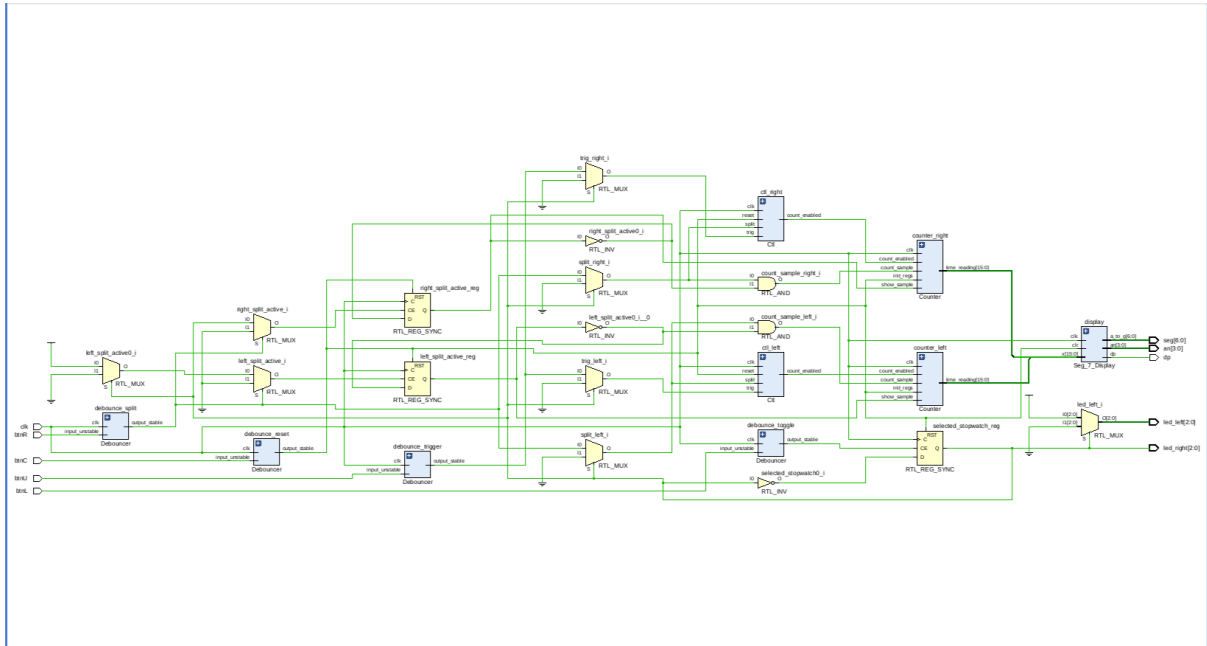


Figure 4: RTL-elaborated schematic of Stopwatch. All four Debouncer instances, the two Ct1 blocks, the two Counter blocks and the shared Seg_7_Display appear as hierarchical boxes. The orange RTL_MUX gates implement the input routing (trig / split go only to the selected side), RTL_AND gates produce the count_sample pulses (split & ~split_active), and the synchronous RTL_REG_SYNC elements are the selected_stopwatch, left_split_active and right_split_active registers. All top-level ports (btnC, btnU, btnR, btnL, clk on the left and seg[6:0], an[3:0], dp, led_left[2:0], led_right[2:0] on the right) are present.

- **Synthesis:**

6c.i Synthesized schematic.

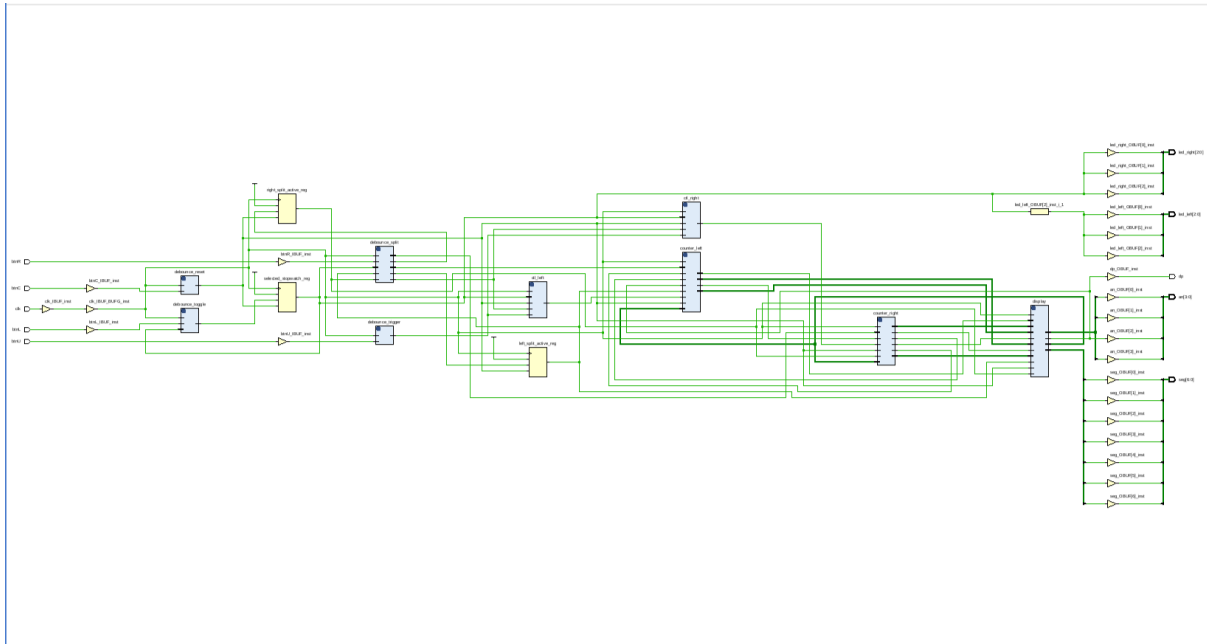


Figure 5: Post-synthesis schematic of `Stopwatch`. The hierarchical boxes are still recognisable, but every external signal now passes through an IBUF / OBUF I/O buffer, the clock goes through a BUFG global buffer, and the combinational logic inside each block has been mapped onto LUTs and FDRE/FDSE flip-flops. The `led_left` / `led_right` OBUFs on the right show that synthesis recognised that the three LEDs of each side share the same drive signal (`selected_stopwatch` or its inverse).

6c.ii Differences between elaborated and synthesized. In the elaborated design, we see the logical/behavioral structure as written in Verilog—high-level modules, generic RTL primitives (RTL_MUX, RTL_REG, RTL_AND), and abstract signal buses. After synthesis, Vivado maps the design onto actual FPGA primitives: LUTs (Look-Up Tables), FDREs/FDSEs (flip-flops with reset/set and enable), IBUFs/OBUFs (I/O buffers), and a BUFG (global clock buffer). The synthesizer also performs optimizations such as merging logic, removing unused paths, and flattening hierarchies where beneficial.

Implementing the design:

8c.i Timing summary.

Setup			Hold			Pulse Width		
Worst Negative Slack (WNS):	5.301 ns		Worst Hold Slack (WHS):	0.212 ns		Worst Pulse Width Slack (WPWS):	4.500 ns	
Total Negative Slack (TNS):	0.000 ns		Total Hold Slack (THS):	0.000 ns		Total Pulse Width Negative Slack (TPWS):	0.000 ns	
Number of Failing Endpoints:	0		Number of Failing Endpoints:	0		Number of Failing Endpoints:	0	
Total Number of Endpoints:	422		Total Number of Endpoints:	422		Total Number of Endpoints:	210	
All user specified timing constraints are met.								

Figure 6: Design Timing Summary after implementation. All user-specified timing constraints are met: the Setup, Hold and Pulse-Width slacks are all positive, with zero failing endpoints in every category.

The timing summary reports:

- **Setup:** Worst Negative Slack (WNS) = 5.301 ns – positive $\Rightarrow$ constraint met.
- **Hold:** Worst Hold Slack (WHS) = 0.212 ns – positive $\Rightarrow$ constraint met.
- **Pulse Width:** Worst Pulse-Width Slack (WPWS) = 4.500 ns – positive $\Rightarrow$ constraint met.
- Number of Failing Endpoints: 0 in every category (out of 422 setup/hold endpoints and 210 pulse-width endpoints).

8c.ii Critical setup path length.

Working from the WNS first, the setup slack is the gap between the clock period and the longest data path:

$$\text{slack}_{\text{setup}} = T_{\text{clk}} - T_{\text{critical}} - T_{\text{uncertainty}}$$

$$5.301 \text{ ns} \approx 10 \text{ ns} - T_{\text{critical}}$$

$$T_{\text{critical}} \approx 4.699 \text{ ns.}$$

Vivado's Intra-Clock Paths report confirms this directly:

Name	Slack	Levels	Routes	High Fanout	From	To	Total Delay	Logic Delay	Net Delay	Requirement	Source Clock	Destination Clock	Exc
Path 1	5.301	4	4	10	counter_left/clk_cnt_reg[19]/C	counter_left/on...seconds_reg[0]/D	4.743	1.212	3.531	10.0	sys_clk_pin	sys_clk_pin	
Path 2	5.315	3	4	10	counter_left/clk_cnt_reg[19]/C	counter_left/ten...seconds_reg[0]/D	4.389	1.061	3.328	10.0	sys_clk_pin	sys_clk_pin	
Path 3	5.402	4	4	10	counter_left/clk_cnt_reg[19]/C	counter_left/ten...seconds_reg[1]/D	4.630	1.202	3.428	10.0	sys_clk_pin	sys_clk_pin	
Path 4	5.415	4	4	10	counter_left/clk_cnt_reg[19]/C	counter_left/ten...seconds_reg[2]/D	4.615	1.202	3.413	10.0	sys_clk_pin	sys_clk_pin	
Path 5	5.436	4	4	10	counter_left/clk_cnt_reg[19]/C	counter_left/ten...seconds_reg[3]/D	4.596	1.202	3.394	10.0	sys_clk_pin	sys_clk_pin	
Path 6	5.461	4	4	10	counter_left/clk_cnt_reg[19]/C	counter_left/ten...seconds_reg[4]/D	4.608	1.214	3.394	10.0	sys_clk_pin	sys_clk_pin	
Path 7	5.486	4	4	9	counter_right/clk_cnt_reg[10]/C	counter_right/ten...seconds_reg[1]/D	4.474	1.425	3.049	10.0	sys_clk_pin	sys_clk_pin	
Path 8	5.488	4	4	9	counter_right/clk_cnt_reg[10]/C	counter_right/ten...seconds_reg[0]/D	4.471	1.425	3.046	10.0	sys_clk_pin	sys_clk_pin	
Path 9	5.513	4	4	9	counter_right/clk_cnt_reg[10]/C	counter_right/ten...seconds_reg[2]/D	4.483	1.437	3.046	10.0	sys_clk_pin	sys_clk_pin	
Path 10	5.517	4	4	9	counter_right/clk_cnt_reg[10]/C	counter_right/ten...seconds_reg[3]/D	4.479	1.430	3.049	10.0	sys_clk_pin	sys_clk_pin	

Figure 7: Intra-Clock Paths – Setup. The critical setup path (Path 1) runs from counter_left/clk_cnt_reg[19]/C to counter_left/on...seconds_reg[0]/D, with a total datapath delay of 4.743 ns (1.212 ns logic + 3.531 ns net), 4 logic levels and a slack of 5.301 ns against the 10 ns requirement.

The reported total path length is therefore $T_{\text{critical}} = 4.743 \text{ ns}$. The very small difference from the 4.699 ns derived from the slack is the clock uncertainty/jitter term that Vivado subtracts separately. For completeness, the worst hold path is shown below:

Intra-Clock Paths - sys_clk_pin - Hold													
Name	Slack	Levels	Routes	High Fanout	From	To	Total Delay	Logic Delay	Net Delay	Requirement	Source Clock	Destination Clock	E
Path 11	0.212	1	1	4	ctl_rightstate_reg[0]C	ctl_rightstate_reg[2]D	0.389	0.255	0.134	0.0	sys_clk_pin	sys_clk_pin	
Path 12	0.222	1	1	1	counter_left[sa..._reg[14]C	displaydigit_reg[2]D	0.367	0.255	0.112	0.0	sys_clk_pin	sys_clk_pin	
Path 13	0.236	1	1	1	debounce_toggle_rev_msb_regC	debounce_toggle_stable_regD	0.395	0.329	0.066	0.0	sys_clk_pin	sys_clk_pin	
Path 14	0.236	1	1	1	debounce_splitrev_msb_regC	debounce_split_1_stable_regD	0.395	0.329	0.066	0.0	sys_clk_pin	sys_clk_pin	
Path 15	0.241	1	1	1	counter_left[sa..._reg[1]C	displaydigit_reg[3]D	0.363	0.255	0.128	0.0	sys_clk_pin	sys_clk_pin	
Path 16	0.241	1	1	1	debounce_reset_rev_msb_regC	debounce_reset_1_stable_regD	0.365	0.305	0.060	0.0	sys_clk_pin	sys_clk_pin	
Path 17	0.250	1	1	1	debounce_toggle_stable_regC	selected_stopwatch_regD	0.391	0.285	0.106	0.0	sys_clk_pin	sys_clk_pin	
Path 18	0.250	1	1	1	counter_right[sa..._reg[12]C	displaydigit_reg[0]D	0.391	0.285	0.106	0.0	sys_clk_pin	sys_clk_pin	
Path 19	0.269	1	1	4	ctl_rightstate_reg[2]C	ctl_rightstate_reg[0]D	0.413	0.285	0.128	0.0	sys_clk_pin	sys_clk_pin	
Path 20	0.275	1	1	10	counter_left[on...conds_reg[0]C	counter_left[on...conds_reg[1]D	0.434	0.329	0.105	0.0	sys_clk_pin	sys_clk_pin	

Figure 8: Intra-Clock Paths – Hold. The worst hold path (Path 11) runs inside `ctl_right` between two adjacent state-register bits, with a total delay of 0.389 ns and a positive slack of 0.212 ns, confirming that no flip-flop violates its hold time.

FPGA programming and debugging:

9a-b. On hardware the two stopwatches behave as specified:

- `btnL` toggles the selection between the left and right stopwatches; the `led_left` / `led_right` bars indicate which side is active.
- `btnU` (trig) starts and pauses *only* the currently selected stopwatch; the other side keeps running unaffected.
- `btnR` (split) freezes / unfreezes the displayed value of the currently selected side without stopping the underlying counter.
- `btnC` (reset) zeroes both stopwatches and brings the selection back to the left side.

9c. Post-implementation project summary. After verifying the design on hardware, the debug core was removed and the design was re-implemented to obtain the final post-implementation reports:

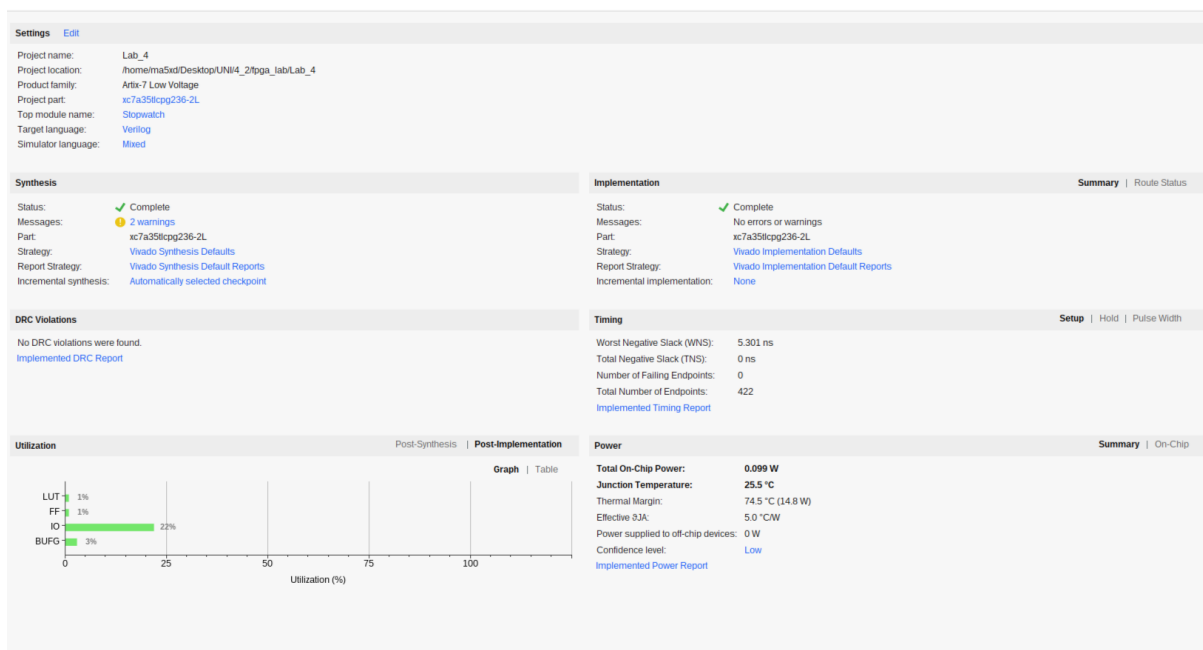


Figure 9: Project Summary after implementation

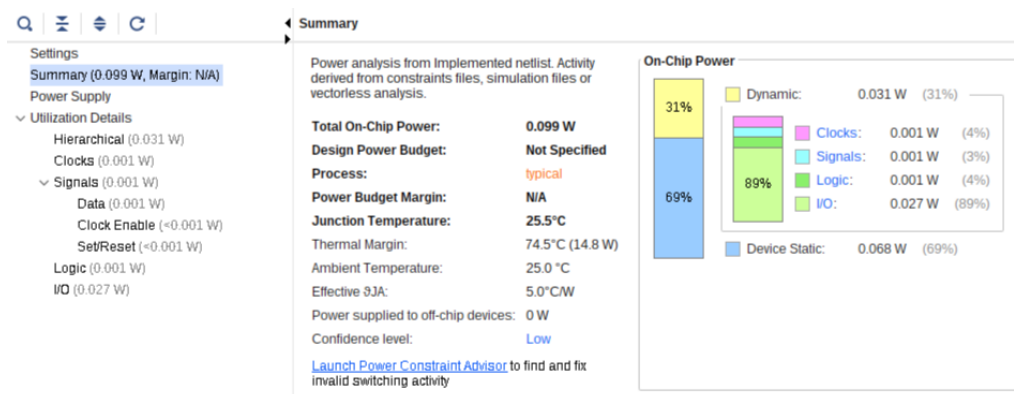


Figure 10: Power report. Total on-chip power is 0.099 W, of which 0.031 W (31%) is dynamic (mostly I/O at 0.027 W) and 0.068 W (69%) is device static. Junction temperature is 25.5 °C with a thermal margin of 74.5 °C (14.8 W).

Appendix – Verilog Source Code

Ctl.v

```
1  `timescale 1ns/10ps
2  module Ctl(clk, reset, trig, split, init_regs, count_enabled);
3
4      input  clk, reset, trig, split;
5      output init_regs, count_enabled;
6
7      localparam SIZE = 3;
8      localparam IDLE = 3'b001, COUNTING = 3'b010, PAUSED = 3'b100;
9      reg [SIZE-1:0] state;
10
11     always @(posedge clk) begin
12         if (reset)
13             state <= IDLE;
14         else begin
15             case (state)
16                 IDLE      : state <= (trig)? COUNTING : IDLE;
17                 COUNTING  : state <= (trig)? PAUSED   : COUNTING;
18                 PAUSED   : begin
19                     casez({reset, trig, split})
20                         3'b000 : state <= PAUSED;
21                         3'b001 : state <= IDLE;
22                         3'b01? : state <= COUNTING;
23                         3'b1?? : state <= IDLE;
24                         default: state <= PAUSED;
25                     endcase
26                 end
27                 default : state <= IDLE;
28             endcase
29         end
30     end
31
32     assign init_regs      = (state == IDLE);
33     assign count_enabled = (state == COUNTING);
34
35 endmodule
```

Ctl_tb.v

```

1 module Ctl_tb();
2     reg clk, reset, trig, split, correct, loop_was_skipped;
3     wire init_regs, count_enabled;
4     integer ai, cii;
5     localparam IDLE = 3'b001, COUNTING = 3'b010, PAUSED = 3'b100;
6     Ctl uut(.clk(clk), .reset(reset), .trig(trig), .split(split),
7             .init_regs(init_regs), .count_enabled(count_enabled));
8     initial begin
9         correct = 1; loop_was_skipped = 0;
10        ai = 0; cii = 0;
11        clk = 0; reset = 1; trig = 0; split = 0;
12        #10 reset = 0;
13        correct = correct & init_regs & ~count_enabled;
14        @(posedge clk);
15        loop_was_skipped = 1;
16        for (ai = 0; ai < 3; ai = ai + 1) begin
17            loop_was_skipped = 0;
18            for (cii = 0; cii < 8; cii = cii + 1) begin
19                reset = 1; #10 reset = 0;
20                if (ai != IDLE) begin trig = 1; #10 trig = 0; end
21                if (ai == PAUSED) begin trig = 1; #10 trig = 0; end
22                {reset, trig, split} = cii;
23                #10
24                {reset, trig, split} = 0;
25                case (ai)
26                    IDLE: casez(cii)
27                        3'b00?: correct = correct & init_regs & ~
28                            count_enabled & (uut.state == IDLE);
29                        3'b11?: correct = correct & init_regs & ~
30                            count_enabled & (uut.state == IDLE);
31                        3'b01?: correct = correct & ~init_regs &
32                            count_enabled & (uut.state == COUNTING);
33                        default: correct = 0;
34                    endcase
35                    COUNTING: casez(cii)
36                        3'b11?: correct = correct & init_regs & ~
37                            count_enabled & (uut.state == IDLE);
38                        3'b00?: correct = correct & ~init_regs &
39                            count_enabled & (uut.state == COUNTING);
40                        3'b01?: correct = correct & ~init_regs & ~
41                            count_enabled & (uut.state == PAUSED);
42                        default: correct = 0;
43                    endcase
44                    PAUSED: casez(cii)
45                        3'b11?: correct = correct & init_regs & ~
46                            count_enabled & (uut.state == IDLE);
47                        3'b01?: correct = correct & ~init_regs &
48                            count_enabled & (uut.state == COUNTING);
49                        3'b001: correct = correct & init_regs & ~
50                            count_enabled & (uut.state == IDLE);
51                        3'b000: correct = correct & ~init_regs & ~
52                            count_enabled & (uut.state == PAUSED);
53                        default: correct = 0;
54                    endcase
55                endcase
56            end
57        end
58        if (correct) $display("Test Passed - %m");
59        else $display("Test Failed - %m");
60        $finish;
61    end

```

```
52     always #5 clk = ~clk;
53 endmodule
```

Debouncer.v

```
1  `timescale 1ns/10ps
2  module Debouncer(clk, input_unstable, output_stable);
3
4      input  clk, input_unstable;
5      output reg output_stable;
6
7      parameter COUNTER_BITS = 7;
8      reg prev_msb;
9      reg [COUNTER_BITS-1:0] counter;
10
11      always @(posedge clk) begin
12          if (input_unstable == 1)
13              counter <= (counter < {COUNTER_BITS{1'b1}}) ? counter + 1 : counter;
14          else
15              counter <= (counter > {COUNTER_BITS{1'b0}}) ? counter - 1 : counter;
16
17          prev_msb      <= counter[COUNTER_BITS-1];
18          output_stable <= (~prev_msb) & counter[COUNTER_BITS-1];
19      end
20
21 endmodule
```

Stopwatch.v

```
1 `timescale 1ns/10ps
2 module Stopwatch(clk, btnC, btnU, btnR, btnL,
3                 seg, an, dp, led_left, led_right);
4
5     input          clk, btnC, btnU, btnR, btnL;
6     output wire [6:0] seg;
7     output wire [3:0] an;
8     output wire      dp;
9     output wire [2:0] led_left;
10    output wire [2:0] led_right;
11
12    wire [15:0] time_reading;
13    wire trig, split, reset, toggle;
14    wire trig_right, split_right, init_regs_right, count_enabled_right,
15          count_sample_right, show_sample_right;
16    wire trig_left, split_left, init_regs_left, count_enabled_left,
17          count_sample_left, show_sample_left;
18
19    localparam LEFT = 1'b0, RIGHT = 1'b1;
20    reg selected_stopwatch;
21    wire [15:0] time_reading_left, time_reading_right;
22    reg right_split_active, left_split_active;
23
24    // Button debouncing
25    Debouncer #(.COUNTER_BITS(20)) debounce_reset (.clk(clk), .
26      input_unstable(btnC), .output_stable(reset));
27    Debouncer #(.COUNTER_BITS(20)) debounce_trigger(.clk(clk), .
28      input_unstable(btnU), .output_stable(trig));
29    Debouncer #(.COUNTER_BITS(20)) debounce_split (.clk(clk), .
30      input_unstable(btnR), .output_stable(split));
31    Debouncer #(.COUNTER_BITS(20)) debounce_toggle (.clk(clk), .
32      input_unstable(btnL), .output_stable(toggle));
33
34    // Controls
35    Ctl ctl_right(.clk(clk), .reset(reset),
36      .trig(trig_right), .split(split_right),
37      .init_regs(init_regs_right), .count_enabled(
38        count_enabled_right));
39    Ctl ctl_left (.clk(clk), .reset(reset),
40      .trig(trig_left), .split(split_left),
41      .init_regs(init_regs_left), .count_enabled(
42        count_enabled_left));
43
44    // Shared 7-segment display
45    Seg_7_Display display(.x(time_reading), .clk(clk), .clr(reset),
46      .a_to_g(seg), .an(an), .dp(dp));
47
48    // Counters
49    Counter counter_right(.clk(clk), .init_regs(reset),
50      .count_enabled(count_enabled_right),
51      .count_sample(count_sample_right),
52      .show_sample(show_sample_right),
53      .time_reading(time_reading_right));
54    Counter counter_left (.clk(clk), .init_regs(reset),
55      .count_enabled(count_enabled_left),
56      .count_sample(count_sample_left),
57      .show_sample(show_sample_left),
58      .time_reading(time_reading_left));
59
60    // Selection logic
61    always @(posedge clk) begin
```

```

56     if (reset) begin
57         selected_stopwatch <= LEFT;
58         left_split_active  <= 1'b0;
59         right_split_active <= 1'b0;
60     end else begin
61         if (toggle) selected_stopwatch <= ~selected_stopwatch;
62         if (split) begin
63             right_split_active <= (selected_stopwatch == RIGHT)? ~
                right_split_active : right_split_active;
64             left_split_active  <= (selected_stopwatch == LEFT) ? ~
                left_split_active  : left_split_active;
65         end
66     end
67 end

68
69 // Routing - LEFT
70 assign led_left      = (selected_stopwatch == LEFT)? 3'b111 : 3'b000;
71 assign trig_left     = (selected_stopwatch == LEFT)? trig   : 1'b0;
72 assign split_left    = (selected_stopwatch == LEFT)? split  : 1'b0;
73 assign show_sample_left = left_split_active;
74 assign count_sample_left = split_left & ~left_split_active;
75
76 // Routing - RIGHT
77 assign led_right     = (selected_stopwatch == RIGHT)? 3'b111 : 3'
    b000;
78 assign trig_right    = (selected_stopwatch == RIGHT)? trig   : 1'b0;
79 assign split_right    = (selected_stopwatch == RIGHT)? split  : 1'b0;
80 assign show_sample_right = right_split_active;
81 assign count_sample_right = split_right & ~right_split_active;
82
83 // Display shares two digits per side (seconds nibbles)
84 assign time_reading = {time_reading_left[15:8], time_reading_right
    [15:8]};
85
86 endmodule

```